

Figure 3.11 Using the darker and brighter functions of a `d3.rgb` object in the highlighting function produces a darker version of the set color for teams from the same region and lighter colors for teams from different regions.

```
function highlightRegion2(d,i) {
 var teamColor = d3.rgb("pink")
 d3.select(this).select("text").classed("highlight", true).attr("y", 10)
 d3.selectAll("g.overallG").select("circle")
       .style("fill", function(p) {return p.region == d.region ?
               teamColor.darker(.75) : teamColor.brighter(.5)})
 this.parentElement.appendChild(this);
}
```

Notice that you can set the intensity for how much brighter or darker you want the color to be. Our new version (shown in figure 3.11) now maintains the palette during highlighting, with darker colors coming to the foreground and lighter colors receding. Unfortunately, you lose the ability to style with CSS because you're back to using inline styles. As a rule, you should use CSS whenever you can, but if you want access to things like dynamic colors and transparency using D3 functions, then you'll need to use inline styling.

You can represent color in other ways with various benefits, but we'll only deal with HSL, which stands for hue, saturation, and lightness. The corresponding `d3.hsl()` allows you to create HSL color objects in the same way that you would with `d3.rgb()`. The reason why you may want to use HSL is to avoid the muddying when you darken pink, which can also happen when you build color ramps and mix colors using D3 functions.

COLOR MIXING

In chapter 2, we mapped a color ramp to numerical data to generate a spectrum of color representing our datapoints. But the interpolated values for colors created by these ramps can be quite poor. As a result, a ramp that includes, say, yellow, can end up interpolating values that are muddy and hard to distinguish. You may think this isn't important, but when you're using a color ramp to indicate a value and your color ramp doesn't interpolate the color in a way that your reader expects, then you can end up showing wrong information to your users. Let's add a color ramp to our `buttonClick` function and use the color ramp to show the same information we did with the radius.

```
var ybRamp = d3.scale.linear()
  .domain([0,maxValue]).range(["yellow", "blue"]);
```

This is the same kind of color ramp we built in chapter 2, using the maxValue we calculated for our circle radius scale.

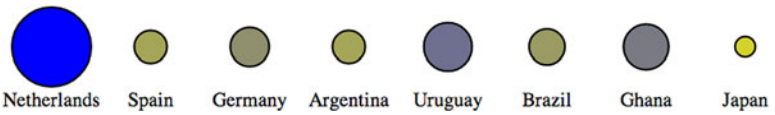


Figure 3.12 Color mixing between yellow and blue in the RGB scale results in muddy, grayish colors displayed for the values between yellow and blue.